

AI6126: Project 1 - CelebAMask Face Parsing Report

Cui Boning [G2403305J]

✉ CUI002@e.ntu.edu.sg

March 28, 2025

1 Preprocessing

The data preprocessing pipeline is designed to standardize both images and masks for effective training:

- **Images** are converted to tensors first and then normalized using a mean and standard deviation of 0.5 to center the data around zero as a common data initialization for better model performance.
- **Masks** are transformed from PIL images to tensors, squeezed to remove the extra channel, and converted to long integers to retain the segmentation labels for cross entropy computation later.

Code is shown below:

```
...
image_transforms = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

mask_transforms = transforms.Compose([
    transforms.PILToTensor(),
    lambda x: x.squeeze(0), # torch.Size([1, 512, 512]) into ([512,
    ↪ 512])
    lambda x: x.long()
])
...
```

A visual comparison can be seen in Figure 1.

2 Model

2.1 Overview

The model adopts a U-Net-like architecture featuring an encoder-decoder structure with skip connections.

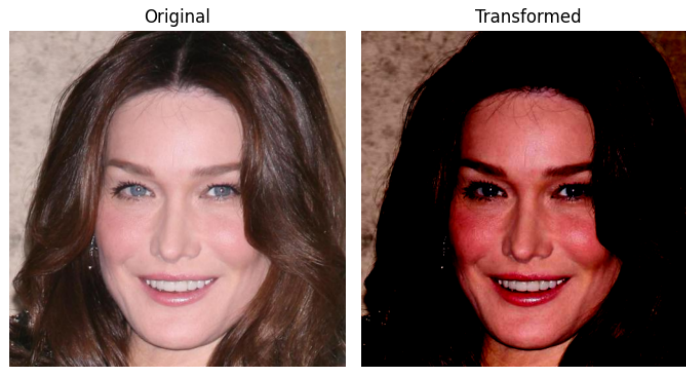


Figure 1: Before and after image transformation

In the encoder, information is progressively downsampled using dilated convolutions and max pooling, capturing broader spatial context without increasing the number of parameters significantly.

At the bottleneck, a multi-scale pyramid pooling module aggregates features at various resolutions, enriching the representation with both local and global details.

The decoder then employs transposed convolutions for upsampling, seamlessly merging high-resolution features from earlier layers via skip connections to prevent information loss.

Overall, this model leverages techniques such as dilated convolutions, multi-scale pyramid pooling, and a U-Net architecture to perform face parsing with a manageable model size. It contains a total number of **1,666,075** trainable parameters.

2.2 Components

- **Encoder Stage:** DualDilatedBlock modules use two parallel dilated convolutions with different dilation rates to expand the receptive field, capturing broader spatial context. This helps differentiate fixed facial regions—ensuring, for example, that areas above the eyes aren’t confused with the neck—while keeping parameter growth in check.

```
class DualDilatedBlock(nn.Module):
    def __init__(self, in_channels, out_channels, dilation1=1,
        ↪ dilation2=1):
        super().__init__()
        mid_channels = out_channels // 4

        self.conv1 = nn.Sequential(
            nn.Conv2d(in_channels, mid_channels, 3,
                ↪ padding=dilation1,
                    dilation=dilation1, bias=False),
            nn.BatchNorm2d(mid_channels),
            nn.ReLU()
        )
        self.conv2 = nn.Sequential(
            nn.Conv2d(in_channels, mid_channels, 3,
                ↪ padding=dilation2,
```

```

        dilation=dilation2, bias=False),
        nn.BatchNorm2d(mid_channels),
        nn.ReLU()
    )

    self.fusion = nn.Sequential(
        nn.Conv2d(mid_channels*2, out_channels, 1,
            ↪ bias=False),
        nn.BatchNorm2d(out_channels)
    )

    self.shortcut = nn.Sequential(
        nn.Conv2d(in_channels, out_channels, 1, bias=False),
        nn.BatchNorm2d(out_channels)
    ) if in_channels != out_channels else nn.Identity()

    def forward(self, x):
        feat1 = self.conv1(x)
        feat2 = self.conv2(x)
        out = torch.cat([feat1, feat2], dim=1)
        out = self.fusion(out)
        out = F.relu(out + self.shortcut(x))
        return out

```

- **Intermediate Stage:** The EnhancedPyramidPool module applies adaptive average pooling at multiple scales, extracting both global and local features. These multi-scale features are upsampled and fused with the original input, enriching the network's overall representation for improved parsing.

```

class EnhancedPyramidPool(nn.Module):
    def __init__(self, in_channels):
        super().__init__()
        reduced_channels = in_channels // 8

        self.paths = nn.ModuleList([
            nn.Sequential(
                nn.AdaptiveAvgPool2d(size),
                nn.Conv2d(in_channels, reduced_channels, 1,
                    ↪ bias=False),
                nn.BatchNorm2d(reduced_channels),
                nn.ReLU()
            ) for size in [1, 2, 4]
        ])

        self.fusion = nn.Sequential(

```

```

        nn.Conv2d(in_channels + reduced_channels*3,
        ↪ in_channels, 1, bias=False),
        nn.BatchNorm2d(in_channels),
        nn.ReLU()
    )

    def forward(self, x):
        h, w = x.shape[2:]
        outputs = [x]

        for path in self.paths:
            feat = path(x)
            outputs.append(F.interpolate(feat, (h, w),
            ↪ mode='bilinear'))

        out = torch.cat(outputs, dim=1)
        return self.fusion(out)

```

- **Decoder Stage:** Transposed convolutions gradually upsample the feature maps to restore the original resolution. Skip connections merge high-resolution encoder features with the upsampled outputs, preserving fine spatial details crucial for precise face segmentation.

```

...
def forward(self, x):
    # Encoder
    x1 = self.conv1(x)           # [B, 24, 256, 256]
    e1 = self.encoder1(x1)       # [B, 24, 256, 256]
    e2 = self.encoder2(e1)       # [B, 48, 128, 128]
    e3 = self.encoder3(e2)       # [B, 96, 64, 64]
    e4 = self.encoder4(e3)       # [B, 144, 32, 32]
    e5 = self.encoder5(e4)       # [B, 192, 16, 16]

    # PPM at deeper level
    ppm = self.ppm(e5)           # [B, 192, 16, 16]

    # Decoder with skip connections
    d5 = self.decoder5(ppm)       # [B,
    ↪ 144, 32, 32]
    d4 = self.decoder4(torch.cat([d5, self.refine4(e4)], 1)) #
    ↪ [B, 96, 64, 64]
    d3 = self.decoder3(torch.cat([d4, self.refine3(e3)], 1)) #
    ↪ [B, 48, 128, 128]
    d2 = self.decoder2(torch.cat([d3, self.refine2(e2)], 1)) #
    ↪ [B, 24, 256, 256]

```

```

d1 = self.decoder1(torch.cat([d2, self.refine1(e1)], 1)) #
    ↪ [B, 24, 512, 512]

return self.final(d1)          # final per-pixel logits output
    ↪ [B, 19, 512, 512]

...

```

3 Hyperparameters

- epoch = 300.
- batch size = 4.
- criterion = nn.CrossEntropyLoss(). Weighted CE was also experimented to address the imbalanced class distribution—since classes like hair, face, and background have much higher pixel frequencies than other classes—but somehow it performed worse than penalizing all classes equally in the loss.
- optimizer = optim.Adam(model.parameters(), lr=0.001), which somehow outperformed SGD by nearly 10% in F1 score over training dataset when model converged.
- scheduler = lr_scheduler.StepLR(optimizer, step_size=25, gamma=0.2) for better convergence.

4 Codabench Evaluation

The best F-measure on the test set on Codabench is 0.81, the screenshot is shown in Figure 2.

ID # ▾	File name	Date	Status	Score	Actions
253742	last_go.zip	2025-03-27 21:10	Finished	0.81	✓

Figure 2: Codabench best score